```
{
    int expected=1;
    while(!sync.compare_exchange_strong(expected,2,
                                        std::memory_order_acq_rel))
        expected=1;
}
void thread_3()
{
    while(sync.load(std::memory_order_acquire)<2);
    // ...
}
```

If you use read-modify-write operations, it's important to pick which semantics you desire. In this case, you want both acquire and release semantics, so `memory_order_acq_rel` is appropriate, but you can use other orderings too. A `fetch_sub` operation with `memory_order_acquire` semantics doesn't synchronize-with anything, even though it stores a value, because it isn't a release operation. Likewise, a store can't synchronize-with a `fetch_or` with `memory_order_release` semantics, because the read part of the `fetch_or` isn't an acquire operation. Read-modify-write operations with `memory_order_acq_rel` semantics behave as both an acquire and a release, so a prior store can synchronize-with such an operation, and it can synchronize-with a subsequent load, as is the case in this example.

If you mix acquire-release operations with sequentially consistent operations, the sequentially consistent loads behave like loads with acquire semantics, and sequentially consistent stores behave like stores with release semantics. Sequentially consistent read-modify-write operations behave as both acquire and release operations. Relaxed operations are still relaxed but are bound by the additional synchronizes-with and consequent happens-before relationships introduced through the use of acquire-release semantics.

Despite the potentially non-intuitive outcomes, anyone who's used locks has had to deal with the same ordering issues: locking a mutex is an acquire operation, and unlocking the mutex is a release operation. With mutexes, you learn that you must ensure that the same mutex is locked when you read a value as was locked when you wrote it, and the same applies here; your acquire and release operations have to be on the same variable to ensure an ordering. If data is protected with a mutex, the exclusive nature of the lock means that the result is indistinguishable from what it would have been had the lock and unlock been sequentially consistent operations. Similarly, if you use acquire and release orderings on atomic variables to build a simple lock, then from the point of view of code that *uses* the lock, the behavior will appear sequentially consistent, even though the internal operations are not.

If you don't need the stringency of sequentially consistent ordering for your atomic operations, the pair-wise synchronization of acquire-release ordering has the potential for a much lower synchronization cost than the global ordering required for sequentially consistent operations. The trade-off here is the mental cost required to ensure that the ordering works correctly and that the non-intuitive behavior across threads isn't problematic.

Data Dependency with Acquire-Release Ordering and Memory_Order_Consume